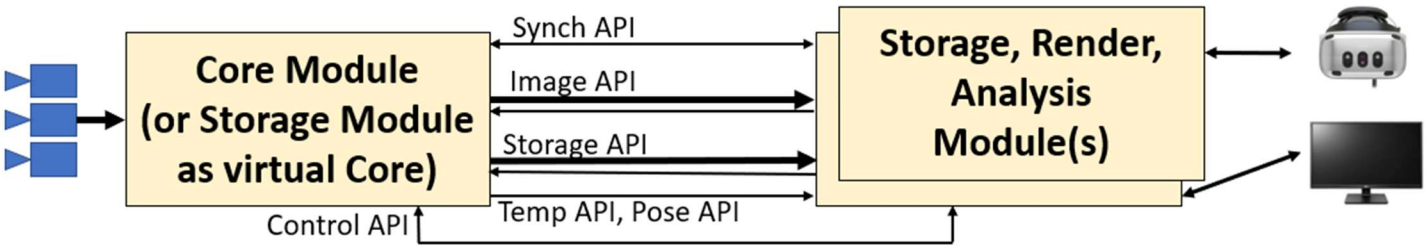
API Implementations

This document describes the implementation design for the Render Module for the Apache IR strap-down pilotage project. It is based on the architecture described in the *TR006v06_Software_Architecture* document along with decisions made through 10/2/2024. Version 6 adds TCP transport, changing the API considerably (Core version 2.0.0).

APIs architecture



The diagram above, from *TR006v06_Software_Architecture*, shows the available Core Module APIs. That document also describes the UDP packet formats and system behavior for each API. (The same document describes the Analysis API, leaving any transport of its JSON strings up to the implementation. Its implementation is described in a separate *API Implementation* document.)

Design decisions

- **Cross platform:** When available, cross-platform (Windows/Linux, i86/ARM) libraries and approaches will be used.
- **CMake:**[1] This cross-platform build system will be used to configure and build the server and client libraries.
- **C++ static:** To minimize application code and to maximize run-time stability, the APIs will be developed in C++ and compiled as a static library.
- **No scatter reads:** The initial architecture does not support scatter-read because of the interleaved messages within packets and the dependency on a message header to know its size. The client memory bandwidth is expected to be sufficient to handle the extra user-space copy. If this becomes a bottleneck, a new version of the architecture will provide a second, streaming, protocol for images that sends all packet sizes ahead of time and then streams packets with the specified sizes, with all packets for a single camera coming to a unique port. We could use the same protocol but enforce exactly one message per packet, predicting the incoming packet sizes.

Notes

- **Interrupt coalescing:** Consider adjusting interrupt coalescing parameters for the NIC to improve throughput.
- **Jumbo packets:** Set these to be as large as possible given the FPGA and client NIC/driver capabilities.

[1] https://cmake.org

Testing

See document *TR-012v08_System_Testing_Design.docx* for an overview of tests that were run on the system.

As of 10/2/204, throughput testing has been completed for sending 25 cameras at 60fps from a camera simulator to a Render Module (“live” mode). Although some UDP packets are out of order, none are lost, and they are sorted before being processed. After the clock synchronization has taken place, there is no jitter in playback for a smoothly-scrolling pattern on all cameras.

As of 10/2/2024, throughput testing has been completed for sending 25 cameras at 60fps from a camera simulator running on one system to a Storage Module running on another over 100GigE (“store” mode). Although some UDP packets are out of order, none are lost. The Storage Module re-orders the packets before sending.

As of 10/2/2024, throughput testing has been completed for sending 25 cameras at 60fps from a Storage Module to a Render Module (“replay” mode). Although some UDP packets are out of order, none are lost, and they are sorted before being processed.

API implementations

The various APIs share some common behaviors (packet and message formats, packet size limitations, sending and receiving UDP packets with specified endpoints, packing and unpacking messages from packets, event type definitions, etc.). Each API has its own specific message-handling behaviors.

Both the common and API-specific behaviors for all APIs are implemented in a cross-platform manner in the ***ASDP*** library. It also implements the Timing submodule that is used by several modules.

Common implementation

Definitions

There are definitions for constants used by the library:

Operation codes will be an **asdp::OpCode** enum with the following values:

- RESET (0)
- CANCEL_ALL_STREAMS (1)
- START_RECORDING (2)
- STOP_RECORDING (3)
- START_REPLAY (4)
- PAUSE_REPLAY (5)
- RESUME_REPLAY (6)
- STOP_REPLAY (7)
- SET_START_UP_RECORDING_STATE (8)
- SET_STATE_PERIOD (10)
- CONFIGURE_TRIGGER (10000)
- SOFTWARE_TRIGGER (10001)
- SET_EVENT_VERBOSITY (10002)
- STREAM_SUBREGION (20000)
- CANCEL_SUBREGION (20001)
- LIST_STORED_STREAMS (30001)
- ERASE_STORED_STREAM (30002)
- STREAM_TEMPERATURES (40000)
- CANCEL_TEMPERATURES (40001)
- STREAM_POSES (50000)
- CANCEL_POSES (50001)

Messages will be an **asdp::MessageID** enum with the following values:

- DISCOVERY (0)
- STATE (1)
- EVENT (10000)
- FRAME_BEGIN (20000)
- FRAME_DATA (20001)
- FRAME_END (20002)
- STREAM_LIST (30000)
- TEMPERATURE (40000)
- POSE (50000)

Events will be an **asdp::EventID** enum with the following values:

- INVALID_OPERATION (256)
- INTERNAL_ERROR (257)
- UNRECOGNIZED_OPCODE (512)
- CLOCK_SYNC (768)
- START_OF_REPLAY (769)
- END_OF_REPLAY (770)
- NUC_FLAG_STATE (771)
- ON_CAMERA_NUC_STATE (772)
- REPLAY_PAUSED (773)
- REPLAY_RESUMED (774)

Features will be an **asdp::FeatureID** enum with the following values:

- STORAGE_API_AVAILABLE (1)
- TEMPERATURE_API_AVAILABLE (3)
- POSE_API_ORIENTATION_AVAILABLE (4)
- POSE_API_POSITION_AVAILABLE (5)
- NUC_FLAG_AVAILABLE (6)
- ON_CAMERA_NUC_AVAILABLE (7)

Library functions and methods will return values from an **asdp::Status** enum:

- OKAY (0)
- TIMEOUT (1)
- THREAD_COMPLETED (2)
- HIGHEST_WARNING (1000)
- BAD_PARAMETER (1001)
- OUT_OF_MEMORY (1002)
- NOT_IMPLEMENTED (1003)
- DELETION_FAILED (1004)
- NULL_OBJECT_POINTER (1005)
- INTERNALEXCAPTION (1006)
- SOCKET_ERROR (1007)
- READ_PAST_END (1008)
- BAD_COOKIE (1009)
- WRITE_PAST_END (1010)
- SOCKET_READ_FAILURE (1011)
- FILE_FAILURE (1012)
- UNEXPECTED_INTERNAL_STATE (1013)
- INCORRECT_ENDIANNESS (1014)
- NOT_CONNECTED (1015)
- INCORRECT_FLOAT_SIZE (1016)
- INCOMPATIBLE_API_VERSION (1017)

Types

Each object type is a class. There is an associated *Get* function for each parameter. Some types have a "base type" that has a *Get* function for its (operation code or message type) that a receiver can use to determine it so that it knows which functions to call for this specific type of object.

General functions and types

Note: All library functions return an asdp::Status value indicating whether they worked. Constructors set a value that can be read using a GetConstructorStatus() method.

There is a base **asdp::Core** class, which forms the basis for both client-side and server-side operation. There are some methods that are implemented in this base class and others (including the constructors) implemented in the derived client and server classes. The base-class methods include:

- asdp::Timer()
    - A reference to this class (which implements the Timer submodule) is exposed by each asdp::Core object, for use by the application. It is built into this API because it learns what it needs from the incoming packets, using a separate thread to ensure fast reads on the clock packets. This is also used internally to determine the time when a packet is sent. On the server side, it can use an offset from its time of construction using the chrono::steady_clock() because the FPGA won't be using the library. On the client side, it provides a method to translate from local-clock time into the server-side local time.
- asdp::Core::GetMaximumPayloadSize(unsigned *value) (applies to UDP transport)
- asdp::Core::SetMaximumPayloadSize(unsigned value) (applies to UDP transport)

The following basic types are available, with appropriate Create/Destroy and access methods:

- asdp::SenderReceiverTCP(std::string host, uint16_t port)
- asdp::SenderUDP(std::string host, uint16_t port, bool broadcast = false)
- asdp::ReceiverUDP(uint32_t IP, uint16_t port)
    - Port 0 means "any available"; GetPort() will return the assigned port

API first-class types

The complete set of creation parameters and get arguments are listed for one command type below to provide a detailed example.

- asdp::CommandPacket {base type read from file or socket before it has a known type}
    - asdp::CommandPacket::GetOpcode(uint32_t &value)
- asdp::CommandPacketStreamTemperatures(uint32_t IP, uint16_t Port, float Period) {client side}
    - asdp::CommandPacket::GetIP(uint32_t *value)
    - asdp::CommandPacket::GetPort(uint16_t *value)
    - asdp::CommandPacket::GetPeriod(float *value)
- asdp::CommandPacketStreamTemperatures(const asdp::CommandPacket &p) {server side}
    - This offers a way to type-case the base class into a subclass once its opcode is known.
    - This will re-use the same in-memory storage but add new methods that point to additional offsets in memory where the subclass values are stored.

There are similar constructors and Get routines for all other CommandPacket derived classes.

There are similar Create and Get methods for the **asdp::StreamPacket** and **asdp::Message** base type and its derived types for each message type.

There are additional helper functions for some first-class types, described in the client-side and server-side sections.

Client-side implementation

The following additional functions are available:

- asdp::CoreClient(NICName)
  - Waits for Discovery broadcasts to determine where to send packets.
  - A name that starts with file:// means to launch a local device reading from a file whose local streaming-in packet receivers will internally shuffle from disk reads. The name in this case is the root directory holding the stored streams by ID.
- asdp::CoreClient::IdentifiedServers(std::vector<std::string> &serverURLs)
  - Becomes non-empty after a Discovery packet has been received from one or more servers.
- asdp::CoreClient::ConnectToServer(std::string serverURL)
  - Name received from IdentifiedServers.
- asdp:CoreClient::GetServerSerialNumber(uint32_t &serial)
  - Set once we have connected to a server, based on the discovery packet information.
- asdp::CoreClient::GetMyIP(uint32_t *value)
  - IP address corresponding to the string name system was opened with.
- asdp::CoreClient::SendCommandPacket(const CommandPacket &packet)
  - Determines where to send the packet based on the connected server.
- asdp::ReceiverUDP::ReceiveStreamPacket(double timeout_seconds, std::shared_ptr<asdp::StreamPacket> *streamPacket)
  - Returns nullptr if no packet available within the specified timeout.
  - Checks the sequence numbers and reports an error if there has been a gap.
- asdp::StreamPacket::GetNextMessage(std::shared_ptr<asdp::Message const> *message)
  - Traverses the messages in the packet until the end of the packet is reached.
  - Fills in a nullptr after the last message in the packet.
  - The data pointer for the image class returns a pointer rather than a copy of the data.
- asdp::Message::GetType(uint32_t &value)
- asdp:: MessageState(Message& baseMessage)
  - Also, cast-contructor functions to convert base to each of the other message types
  - Do not Destroy() the messages; they are destroyed when the StreamPacket is destroyed.

Server-side implementation

The camera-capture Core Module implements the APIs on its FPGA fabric as described in a separate document.

A C server-side library is provided for use by stand-alone software Core Module implementations, such as a mock implementation for testing and a loopback-interface implementation that talks to an external NAS with recorded data. This is also usable by a Storage module.

The following additional type is available:

- asdp::StreamWriter(std::shared_ptr<asdp::Sender> sender, std::shared_ptr<asdp::Timer> timer, uint32_t maxPacketSize)
  - Keeps a shared pointer to a Sender() which may be UDP or file.
  - Has a CurrentPacket() call that gets a reference to the current StreamPacket being filled.
  - Has a Flush() call that sends the current packet and prepares a new one
    - Set the time and sequence number on each StreamPacket before sending.
  - Flushes before destroying

The following additional functions are available:

- asdp::CoreServer(std::string NICName, uint32_t maxPayloadSize)
    - Verifies proper endianness and initializes any required networking interfaces and protocols.
    - Starts Discovery broadcasts on the NIC(s) and opens ports to receive commands.
- asdp::ReceiverUDP::ReceiveCommandPacket(double timeout_seconds, std::shared_ptr<asdp::CommandPacket const> *basePacket)
    - Returns nullptr if no packet available within the specified timeout
- asdp::CommandPacket::GetOpcode(uint32_t &value)
- asdp::CommandPacketStreamTemperatures(asdp::CommandPacket const basePacket)
    - Also, constructor functions to convert base to each of the other command-packet types.
    - The base-class Destroy() is called when we're done with the packet.

Read/write: There are classes for reading and writing packets from files, supporting direct access for the Storage API files. They are derived from a common interface base class with SenderUDP and ReceiverUDP.

- asdp::SenderFile(std::string fileName)
    - Writes a stream to a file.
    - Supports SendStreamPacket()
- asdp::ReceiverFile(std::string filename)
    - Reads a stream from a file.
    - Supports IsPacketAvailable() and ReceiveStreamPacket().

Appendix A: Client-Side Example

This example excludes error checking and destruction of structures and includes pseudo-code sections.

Live flight without recording for simple, unsynchronized streams

- CoreClient cc("NIC_to_find_server_on_dotted_decimal")
- std::vector<string> servers; while (servers.size()==0) { cc. IdentifiedServers(&servers); }
- cc.ConnectToServer(servers[0]);
- cc.GetMyIP(&MyIP);
- cc.SendCommandPacket(CommandPacketStreamStatePeriod(1.0))
- Receiver sr; cc.GetStreamReader(sr);
- std::shared_ptr<StreamPacket> packet(nullptr); while (!packet) { sr.ReceiveStreamPacket(0.1, &packet); }
    - std::shared_ptr<Message> message; sp.GetNextMessage(&message); while (message) {
        - uint32_t type; message->GetType(&type) ;
        - if (type == STATE) {
            - MessageState const stateMessage(*message);
            - Allocate buffers and threads for the number of cameras with appropriate resolutions.
        - sp.GetNextMessage(&message);
- Generate a thread to receive image messages from each camera. Each thread configures and reads:
    - ReceiverUDP ThreadImageReceiver(MyIP, 0)
        - uint16_t StatusPort; ThreadImageReceiver.GetPort(&ThreadImagePort)
    - cc.SendCommandPacket(CommandPacketStreamSubregion({MyIP, ThreadImagePort}, …));
    - std::shared_ptr<StreamPacket> packet;
    - while (true) { ThreadImageReceiver.ReceiveStreamPacket(0.1, &packet); if (packet) {
        - Sort the incoming packets by sequence number and then process each in order:
        - std::shared_ptr<Message> message; packet->GetNextMessage(&message);
        - while (message) { uint32_t type; message->GetType(&type) ;
            - if (type == FRAME_BEGIN) { Clear back buffer for this camera }
            - if (type == FRAME_DATA) { Cast and copy frame data to back buffer }
            - if (type == FRAME_END) { Swap back/front buffers }
    - } };
- In a separate thread, render from the front buffer for each camera with the desired viewport.